Local Semantic Search

High-Performance Deep Learning Architecture

Redefining the Local Search Experience

The Problem: Search Latency vs. Intelligence

- **Traditional Lexical Search:** Fast but "dumb" (Exact keyword matches only).
- **Cloud-Based Embeddings:** Intelligent but "slow" (Network latency) and private data exposure.
- **Naive Local Embeddings:** High CPU/Memory overhead, blocking interactive UX.

Core DL Architecture

we achieves interactive speeds by bringing high-quality embeddings to the edge:

- **Backbone:** `ONNX Runtime`.
- **Runtime:** Fully local inference using **FastEmbed-RS**.
- **Engine:** Rust-native execution with zero remote dependencies.
- **Hybrid Core:** Tantivy (Lexical) + HNSW/ANN (Vector) Fusion.

Why minimal ONNX?

Model	Dimensions	Size	Latency
OpenAI `text-embedding-3-small`	1536	Cloud	~500ms+
all-MiniLM-L6-v2	**384**	**80MB**	**~15ms**

- **Distillation:** We use a "student" model that retains ~95% of the performance of larger BERT models at a fraction of the cost.
- **Lower Dimensionality:** 384 dimensions reduce ANN index memory footprint by 4x compared to 1536-dim models.

Performance Secret: ONNX Optimization

We don't just "run" a model; we optimize for the host silicon:

1. **Quantization:** Weights converted to `INT8` reducing size by 4x and increasing throughput by ~2-3x on CPUs.
2. **SIMD & AVX-512:** Utilizing CPU vector instructions for parallel dot-product calculations.
3. **Graph Optimization:** Fusing layers (e.g., LayerNorm) into single kernels to reduce memory bandwidth bottlenecks.

Context-Aware Chunking

Better than traditional embeddings because we understand structure.

- **Semantic Boundaries:** Instead of fixed-size windows (e.g., 512 tokens), we split on paragraph, heading, and code-block boundaries.
- **Overlapping Windows:** 10-15% overlap ensures context isn't lost at the cut point.
- **Specialized Heuristics:** Different chunking logic for `.rs`, `.toml`, and `.md` to preserve logical units.

Hybrid Score Fusion (RRF)

We use **Reciprocal Rank Fusion (RRF)** to combine the best of both worlds:

- **Lexical:** Catches rare terms, unique IDs, and typos.
- **Semantic:** Catches intent, synonyms, and conceptual matches.
- **Result:** Higher Recall than pure embeddings; higher Precision than pure keywords.

The Result: The "Interactive" Bar

- **Query Embedding:** < 20ms
- **Vector Search (HNSW):** < 5ms
- **Hybrid Fusion:** < 10ms
- **Total Pipeline: ~35ms (P95)**

This is 10x faster than cloud-based alternatives, enabling launcher-style "search-as-you-type" for semantic intent.

Roadmap & Future DL Steps

- [ ] **Hardware Acceleration:** Auto-detection for Vulkan/Metal/CUDA backends.
- [ ] **Custom Fine-Tuning:** Domain-specific adapters for local codebase styles.